



# **UNIVERSIDAD DE MONTEMORELOS**

**Facultad de Ingeniería y Tecnología**

## **Actividad**

Odometría de un robot

**Presentado en cumplimiento parcial de los requisitos de la  
materia de:**

Estándares Web

## **Docente:**

Daniel Neri Ramírez

## **Alumno:**

Giovanni Efraín Ortiz González

**Montemorelos, Nuevo León**

**30 de Abril de 2026**

Como proyecto del módulo de JavaScript/NodeJS se nos pidió simular la odometría de un robot. Los datos que se nos entregaron fueron:

- Tamaño de las ruedas: 140 mm.
- Cantidad de tickets para la vuelta de una rueda: 1450 tickets.
- Condiciones para el aumento de ticks:
  - Que se generara un número aleatorio que se tomaría como referencia para decidir si se aumentaban los ticks.
  - El número generado debía estar entre 1 y 100.
  - Si el número era menor a 50 y era impar, entonces aumentaba un ticket.
  - Si el número era mayor a 50 y par, entonces aumentaba un ticket.

Se pidió que se levantara un servidor usando express y que los tickets para cada rueda del robot se publicaran por separado: /ticketsL y /ticketsR.

Con esos datos, entonces se crearía una página en el puerto 45050 donde se verían reflejados los siguientes datos:

- Las dos ruedas:
  - La derecha girando en sentido horario y la izquierda en sentido antihorario para simular su movimiento hacia adelante.
  - El progreso del giro de las ruedas con respecto a los tickets.
  - El avance en cm de cada rueda.
  - La velocidad de cada rueda.
- El robot:
  - La posición del robot en el mapa:
    - Posición con respecto a x.
    - Posición con respecto a y.
    - Ángulo de movimiento.
  - La velocidad lineal del robot.
  - La velocidad angular del robot.

Lo primero que se hizo fue diseñar la estructura del sistema:

```
Odom/
├── backend/
│   └── serverOdom.js
└── frontend/
    ├── odom_dashboard.html
```

```
|— css/  
|   └─ odom.css  
└─ js/  
    └─ odom.js
```

Se inició npm en la carpeta backend con el comando `npm init -y`. También se instaló express con el comando `npm install express`.

En package.json se agregó el script "start": "node serverOdom.js" para iniciar el servidor con el comando `npm start`.

El código del servidor se realizó de la siguiente manera:

```
1  const { getRandomValues } = require('crypto');  
2  const express = require('express');  
3  const path = require('path');  
4  const app = express();  
5  const PORT = 45050;  
6  |  
7  app.use(express.static(path.join(__dirname, '..', 'frontend')));  
8  
9  app.get('/', (req, res) => res.sendFile(path.join(__dirname, '..', 'frontend', 'odom_dashboard.html')));  
10
```

Se inicializaron las constantes que se van a usar y se especificó la ruta para los archivos frontend que se van a mostrar directamente en el puerto 45050.

De ahí se trabajó en el código para generar los tickets de manera aleatoria siguiendo las condiciones que nos fueron dadas:

```

10 let ticketsL = 0;
11 let ticketsR = 0;
12 |
13 function validarIncremento(num) {
14 | return num < 50 ? (num % 2 !== 0) : (num % 2 === 0);
15 | }
16 |
17 function generarSemilla(x) {
18 | let semilla = x;
19 | return function() {
20 | | semilla = (semilla * 1664525 + 1013904223) % 4294967296;
21 | | return (semilla / 4294967296);
22 | | }
23 | }
24 |
25 function aleatorio(min, max) {
26 | return Math.floor((Math.random() * (max - min + 1)) + min);
27 | }
28 |
29 const randomL = generarSemilla(Math.max(Math.floor(Math.random() * 100), Math.floor(Math.random() * 100)));
30 const randomR = generarSemilla(aleatorio((Math.random() * 100), (Math.random() * 100)));
31 |
32 setInterval(() => {
33 | | const num = Math.floor(randomL() * 100) + 1;
34 | | if (validarIncremento(num)) ticketsL++;
35 | | console.log('Tickets para la rueda izquierda: ' + ticketsL)
36 | }, 50);
37 |
38 setInterval(() => {
39 | | const num = Math.floor(randomR() * 100) + 1;
40 | | if (validarIncremento(num)) ticketsR++;
41 | | console.log('Tickets para la rueda derecha: ' + ticketsR)
42 | }, 50);

```

Se usaron dos funciones, la primera que sigue el método de generar una semilla y la segunda función que usa un algoritmo sencillo de PRNG (pseudo-random number generator) y se jugaron con ambas funciones para producir los valores de los tickets. Para que cada tick se produjera de manera independiente se establecieron dos `setInterval` a 50 ms.

```

44 app.get('/ticketsL', (req, res) => res.json({ ticketsL }));
45 app.get('/ticketsR', (req, res) => res.json({ ticketsR }));
46 |
47 app.listen(PORT, () => {
48 | | console.log('Servidor corriendo en http://localhost:45050);
49 | | })
50 |

```

Finalmente, los valores de los tickets son publicados en las rutas dadas por la función `.get` en `/ticketsL` y `/ticketsR`. Y el servidor es levantado en el puerto 45050.

En el archivo `odom.js` que es el javascript de la página se declararon las constantes y variables (como el ancho del robot de 20cm, el diámetro de las llantas, los tickets por

vuelta, las variables para actualizar la posición, etc.) y se definió la función para la Odometría diferencial:

```
1 // =====
2 //  CONSTANTES DE ODOMETRÍA
3 // =====
4 const TICKS_V    = 1450;                // ticks por vuelta
5 const DIAM_MM    = 140;
6 const CIRC_CM    = (Math.PI * DIAM_MM) / 10; // ≈ 43.982 cm
7 const CM_TICK    = CIRC_CM / TICKS_V;    // ≈ 0.03033 cm/tick
8 const ANCHO_CM   = 20;                  // distancia entre ruedas (cm)
9 const ARC_CIRC   = 2 * Math.PI * 35;    // circunferencia del arco SVG (radio 35) ≈ 219.9
10
11 // =====
12 //  ESTADO
13 // =====
14 let pose        = { x: 0, y: 0, theta: 0 };
15 let prevL       = 0, prevR = 0;
16 let trail       = [{ x: 0, y: 0 }];
17 const SCALE     = 6;                    // px / cm (fijo)
18 let scale       = SCALE;                // zoom ajustable
19 let velWin      = [];                  // ventana de velocidad
20 let vLin        = 0, vAng = 0, vWheelL = 0, vWheelR = 0;
21 let ftL, ftR;                          // timers de flash
22
```

```
46 // =====
47 //  ODOMETRÍA DIFERENCIAL
48 // =====
49 function updateOdom(ticksL, ticksR) {
50   const dL = (ticksL - prevL) * CM_TICK;
51   const dR = (ticksR - prevR) * CM_TICK;
52   prevL = ticksL;
53   prevR = ticksR;
54
55   if (dL === 0 && dR === 0) return;
56
57   const d      = (dL + dR) / 2;
58   const dTheta = (dR - dL) / ANCHO_CM;
59
60   // Actualiza ángulo primero, luego posición con ángulo actualizado
61   pose.theta += dTheta;
62   // Convención: theta=0 → robot avanza en +Y (Norte)
63   pose.x += d * Math.cos(pose.theta + Math.PI / 2);
64   pose.y += d * Math.sin(pose.theta + Math.PI / 2);
65
66   trail.push({ x: pose.x, y: pose.y });
67   if (trail.length > 3000) trail.shift();
68 }
```

Para calcular la distancia del robot se aplicó la fórmula: distancia = (distancia de la rueda izquierda + distancia de la rueda derecha) / 2. Para el ángulo se calculó: ángulo Theta = (distancia de la rueda izquierda - distancia de la rueda derecha) / ancho del robot.

Para calcular las distancias individuales de cada rueda se fueron tomado los tickets de avance de cada intervalo de tiempo (tickets actuales - tickets previos) multiplicados por

el valor en centímetros de cada ticket. Y con esos datos se actualizan los valores del ángulo.

Se añadieron otras funciones para actualizar el mapa de posicionamiento del robot y para las representaciones visuales de las ruedas.

La función principal se determinó para funcionar en un intervalo de 100ms. Por medio de fetch la función extrae los valores de los tickets en /ticketsL y /ticketsR:

```
257 // =====
258 //  FETCH PRINCIPAL (100 ms)
259 // =====
260 async function tick() {
261   try {
262     const [rL, rR] = await Promise.all([
263       fetch('/ticketsL'),
264       fetch('/ticketsR')
265     ]);
266     const { ticketsL } = await rL.json();
267     const { ticketsR } = await rR.json();
268     const now = Date.now();
269
270     // — Velocidad (ventana deslizando 500 ms) —
271     velWin.push({ t: now, L: ticketsL, R: ticketsR });
272     velWin = velWin.filter(v => now - v.t <= 500);
273     if (velWin.length >= 2) {
274       const old = velWin[0], nw = velWin[velWin.length - 1];
275       const dt = (nw.t - old.t) / 1000;
276       if (dt > 0) {
277         const dL = (nw.L - old.L) * CM_TICK;
278         const dR = (nw.R - old.R) * CM_TICK;
279         vWheelL = dL / dt;
280         vWheelR = dR / dt;
281         vLin    = (dL + dR) / 2 / dt;
282         vAng    = (dR - dL) / ANCHO_CM / dt;
283       }
284     }
285   }
```

```

286 // — Odometría —
287 updateOdom(ticketsL, ticketsR);
288
289 // — Animación de ruedas —
290 const degT = 360 / TICKS_V;
291 const rotL = ticketsL * degT;
292 const rotR = ticketsR * degT;
293 document.getElementById('rimL').setAttribute('transform', `rotate(${ -rotL })`); // antihorario
294 document.getElementById('rimR').setAttribute('transform', `rotate(${ rotR })`); // horario
295 setArc(document.getElementById('arcL'), ticketsL);
296 setArc(document.getElementById('arcR'), ticketsR);
297
298 // — Barras de progreso —
299 const pL = (ticketsL % TICKS_V) / TICKS_V * 100;
300 const pR = (ticketsR % TICKS_V) / TICKS_V * 100;
301 document.getElementById('barL').style.width = pL + '%';
302 document.getElementById('barR').style.width = pR + '%';
303
304 // — Datos de rueda —
305 const vueLL = Math.floor(ticketsL / TICKS_V);
306 const vueLR = Math.floor(ticketsR / TICKS_V);
307 const partL = ticketsL % TICKS_V;
308 const partR = ticketsR % TICKS_V;
309 document.getElementById('tL').innerHTML = `${ticketsL} <span>ticks</span>`;
310 document.getElementById('tR').innerHTML = `${ticketsR} <span>ticks</span>`;
311 document.getElementById('revL').textContent =
312 | `${vueLL} vuelta${vueLL !== 1 ? 's' : ''} · ${partL} / ${TICKS_V}`;
313 document.getElementById('revR').textContent =
314 | `${vueLR} vuelta${vueLR !== 1 ? 's' : ''} · ${partR} / ${TICKS_V}`;
315

```

```

316 // — Recorrido —
317 const cml = (ticketsL * CM_TICK).toFixed(2);
318 const cmR = (ticketsR * CM_TICK).toFixed(2);
319 document.getElementById('cml').innerHTML = `${cml}<span class="d-unit"> cm</span>`;
320 document.getElementById('cmR').innerHTML = `${cmR}<span class="d-unit"> cm</span>`;
321 document.getElementById('dtL').textContent = `${ticketsL} ticks`;
322 document.getElementById('dtR').textContent = `${ticketsR} ticks`;
323
324 // — Posición —
325 document.getElementById('valX').innerHTML =
326   `${pose.x.toFixed(2)}<span class="pose-unit"> cm</span>`;
327 document.getElementById('valY').innerHTML =
328   `${pose.y.toFixed(2)}<span class="pose-unit"> cm</span>`;
329 document.getElementById('valT').innerHTML =
330   `${pose.theta.toFixed(4)}<span class="pose-unit"> rad</span>`;
331
332 // — Velocidad —
333 document.getElementById('valV').innerHTML =
334   `${vLin.toFixed(2)}<span class="vel-unit"> cm/s</span>`;
335 document.getElementById('valW').innerHTML =
336   `${vAng.toFixed(4)}<span class="vel-unit"> rad/s</span>`;
337 document.getElementById('subV').textContent =
338   vLin > 0.5 ? 'AVANZANDO' :
339   vLin < -0.5 ? 'RETROCEDIENDO' : 'DETENIDO';
340 document.getElementById('subW').textContent =
341   vAng > 0.005 ? 'GIRANDO IZQ' :
342   vAng < -0.005 ? 'GIRANDO DER' : 'RECTO';
343 document.getElementById('valVL').innerHTML =
344   `${vWheelL.toFixed(2)}<span class="vel-unit"> cm/s</span>`;
345 document.getElementById('valVR').innerHTML =
346   `${vWheelR.toFixed(2)}<span class="vel-unit"> cm/s</span>`;
347

```

```

348 // — Mapa —
349 draw();
350
351 } catch (e) {
352   console.error('Error fetch:', e);
353 }
354 }
355
356 setInterval(tick, 100);
357 tick();

```

Con los valores de la odometría diferencial se calcula la posición del robot en x, y y theta. La función también incluye una ventana deslizante de 500ms para calcular las velocidades de manera suave. Y cierra con la función draw() para actualizar el mapa de posicionamiento.

La página se ve de la siguiente manera:



